

```

from PySide6.QtWidgets import QTreeView, QAbstractItemView, QMenu
from PySide6.QtGui import QStandardItemModel, QStandardItem, QCursor
from PySide6.QtCore import Qt, Signal, Slot, QModelIndex, QSortFilterProxyModel, QPoint, QItemSelectionModel, QEvent
import re

from IndexTextFormatterDelegate import IndexTextFormatterDelegate
from IndexTreePersistence import IndexTreePersistence

# --- Explicit Architectural Data Roles ---
ROLE_PATH = Qt.UserRole          # File path string or raw tag name
ROLE_LINE = Qt.UserRole + 1      # Line number (int)
ROLE_IS_LEAF = Qt.UserRole + 2   # Boolean leaf node flag (True = file, False = folder)
ROLE_COL = Qt.UserRole + 3       # Column number (int)
ROLE_COUNT = Qt.UserRole + 4     # Occurrences count (int) for folders

# Custom Roles to hold the UID structure on the IDn cells
ROLE_UID_DATA = Qt.UserRole      # Dictionary: {'id': int, 'path': str, 'line': int, 'col': int}

class CaseInsensitiveItem(QStandardItem):
    """Custom item helper providing case-insensitive text evaluation based on stripped key text."""

    # Matches macro layouts like \textit{Text} or \textbf{Text} to isolate 'Text'
    _MACRO_PATTERN = re.compile(r'\\\[a-zA-Z]+\{([^\}]+\})\\')

    def __init__(self, text=""):
        super().__init__(text)
        # Compute and normalize the sorting key immediately on creation
        self.sort_key = self._compute_clean_sort_key(text)

    def _compute_clean_sort_key(self, text: str) -> str:
        """
        Extracts the true sort-override key or strips formatting macros to evaluate keys.
        Transforms:
            'apple@\textit{apple}' -> 'apple'
            '\textit{Zebra}'       -> 'zebra'
        """
        # Rule 1: Handle sort override split formatting constraints (key@source)
        if '@' in text:
            # The left side of the split represents the deterministic sorting key
            key_part = text.split('@')[0]
        else:
            key_part = text

        # Rule 2: Clean out LaTeX layout macros if they leak into the key section
        # e.g., converts '\textit{banana}' down to 'banana'
        clean_key = self._MACRO_PATTERN.sub(r'\1', key_part)

        # Rule 3: Strip remaining LaTeX control markers like \string
        clean_key = clean_key.replace(r'\string', '')

        # Return case-insensitive, lowercase stripped output string for alpha matching
        return clean_key.strip().lower()

    def __lt__(self, other):
        """Forces Qt sorting pipelines to evaluate items based on clean text keys."""
        if isinstance(other, CaseInsensitiveItem):
            return self.sort_key < other.sort_key
        return self.text().lower() < other.text().lower()

class IndexTreeView(QTreeView):
    locationRequested = Signal(str, int, int)
    # Create a custom stable click signal that completely bypasses Qt's standard selection signals
    referenceTokenClicked = Signal(object, int) # (proxy_index, click_x)

    def __init__(self, parent=None):
        super().__init__(parent)

        # Initialize the underlying standard item data model (Exactly 2 structural columns)
        self.base_model = QStandardItemModel()
        self.base_model.setHorizontalHeaderLabels(["Index Terms", "References"])
        self.setModel(self.base_model)

        # Lock visual behaviors: select single items instead of full row lines
        self.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
        self.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectItems)
        self.setFocusPolicy(Qt.FocusPolicy.StrongFocus)

        # Turn on sorting behavior and display headers click arrows
        self.setSortingEnabled(True)

        # Configure and bind custom context menu pipelines
        self.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)
        self.customContextMenuRequested.connect(self.handle_context_menu_request)

        # Enable core dynamic sorting layout engines
        self.setSortingEnabled(True)

```

```

self.header().setSortIndicator(0, Qt.AscendingOrder)
self.setEditTriggers(QTreeView.NoEditTriggers)
self.setHeaderHidden(False)
self.setMouseTracking(True)

# Instantiate and attach the formatting delegate internally to Column 0
self.formatting_delegate = IndexTextFormatterDelegate(self)
self.setItemDelegateForColumn(0, self.formatting_delegate)

# Connect event slots
self.clicked.connect(self.handle_reference_click)
self.customContextMenuRequested.connect(self.show_context_menu)

def _get_source_model(self):
    """Safely extracts the absolute underlying data storage model across proxy filters."""
    curr_model = self.model()
    if isinstance(curr_model, QSortFilterProxyModel):
        return curr_model.sourceModel()
    return curr_model

def handle_reference_click(self, index: QModelIndex):
    """
    Intercepts single clicks in Column 1, determines which specific [IDn]
    token was clicked based on character width, and triggers navigation.
    """
    if index.column() == 0:
        return

    model = self.model()
    item = model.itemFromIndex(index) if hasattr(model, 'itemFromIndex') else self._get_source_model().itemFromIndex(index)
    if not item:
        return

    # Extract the references array cached on this row by the controller
    refs_list = item.data(Qt.ItemDataRole.UserRole)
    if not refs_list or not isinstance(refs_list, list):
        return

    # 1. Calculate the exact text position where the user clicked inside the item text cell
    click_pos = self.mapFromGlobal(QCursor.pos())
    visual_x_offset = click_pos.x() - self.columnViewportPosition(1) - 4 # Deduct side padding

    # 2. Extract the full string representation (e.g., "[1] [12] [104]")
    full_text = item.text()
    font_metrics = self.fontMetrics()

    # Iterate character-by-character to map the text position to a string index
    char_index = 0
    accumulated_width = 0
    for i, char in enumerate(full_text):
        accumulated_width += font_metrics.horizontalAdvance(char)
        if accumulated_width >= visual_x_offset:
            char_index = i
            break
    else:
        char_index = len(full_text) - 1

    # 3. Parse out the clicked integer token string using regular expressions
    clicked_id_str = None
    for match in re.finditer(r'\[(\d+)\]', full_text):
        if match.start() <= char_index <= match.end():
            clicked_id_str = match.group(1)
            break

    if not clicked_id_str:
        return

    # 4. Look up the matching tracking dictionary inside our list payload
    for uid in refs_list:
        if str(uid.get("id_token")) == clicked_id_str or str(uid.get("id")) == clicked_id_str:
            path = uid.get("path")
            # Normalize line indices to standard 1-based format
            line = uid.get("line_num", uid.get("line", 1))
            col = uid.get("col_num", uid.get("col", 1))

            print(f"NAVIGATION TARGET discovered: ID [{clicked_id_str}] -> {path} on Line {line}")
            self.locationRequested.emit(str(path), int(line), int(col))
            break

@Slot(QPoint)
def handle_context_menu_request(self, position: QPoint):
    """View Context Handler: Restricts right-click menus strictly to Column 0."""
    proxy_index = self.indexAt(position)
    if not proxy_index.isValid() or proxy_index.column() != 0:
        return

    self.selectionModel().select(proxy_index, QItemSelectionModel.SelectionFlag.ClearAndSelect)

```

```

model = self.model()
raw_model = model.sourceModel() if hasattr(model, 'mapToSource') else model
source_index = model.mapToSource(proxy_index) if hasattr(model, 'mapToSource') else proxy_index

item = raw_model.itemFromIndex(source_index) if hasattr(raw_model, 'itemFromIndex') else None
if not item:
    return

menu = QMenu(self)
action_add_sub = menu.addAction(f"Add Subheading to '{item.text()}'")
action_delete = menu.addAction("Delete Entry Globally")

global_pos = self.viewport().mapToGlobal(position)
selected_action = menu.exec(global_pos)

main_win = self.window()
if selected_action == action_add_sub and hasattr(main_win, 'handle_add_subheading_workflow'):
    main_win.handle_add_subheading_workflow(item)
elif selected_action == action_delete and hasattr(main_win, 'handle_delete_workflow'):
    main_win.handle_delete_workflow(item)

@Slot(list, dict)
def add_index_to_tree(self, parts: list, uid_dict: dict):
    """
    Processes index structural additions based on the updated signature.

    parts: e.g., ["Great Britain", "income inequality"]
    uid_dict: {"id": 44, "path": "/doc.tex", "line": 12, "col": 5}
    """
    model = self._get_source_model()
    parent_item = model.invisibleRootItem()

    # 1. Traverse or build the textual hierarchy strictly inside Column 0
    for level_text in parts:
        found_item = None
        for i in range(parent_item.rowCount()):
            child = parent_item.child(i, 0) # Search strictly in the first column

            # Enforce architectural requirement: case-insensitive structural matching
            if child and str(child.text()).lower() == str(level_text).lower():
                found_item = child
                break

        if not found_item:
            # Materialize new structural heading node using your custom wrapper
            found_item = CaseInsensitiveItem(level_text)
            found_item.setEditable(False)

            from PySide6.QtWidgets import QStyle
            found_item.setIcon(self.style().standardIcon(QStyle.SP_DirIcon))

            # Append a new row layout with only Column 0 populated
            parent_item.appendRow([found_item])

        # Nest deeper under Column 0's item matrix branch
        parent_item = found_item

    # 2. Add the unique IDn token as a horizontal column onto this matching row
    # Grab the row index context and parent relative to the deepest matched node
    row_parent = parent_item.parent() or model.invisibleRootItem()
    row_index = parent_item.row()

    # Scan across the row starting at Column 1 to find the next empty horizontal slot
    target_column = 1
    while row_parent.child(row_index, target_column) is not None:
        target_column += 1

    # Instantiate the clickable IDn text element cell
    idn_display_text = str(uid_dict.get("id"))
    idn_item = CaseInsensitiveItem(idn_display_text)
    idn_item.setEditable(False)

    # Inject the entire UID structure dictionary securely on the UserRole mapping register
    # Custom Table Persistent Schema Offset: Qt.UserRole
    from PySide6.QtCore import Qt
    idn_item.setData(uid_dict, Qt.UserRole)

    # Visual cue indicating this cell is an active index hyperlink reference
    from PySide6.QtWidgets import QStyle
    idn_item.setIcon(self.style().standardIcon(QStyle.SP_FileIcon))

    # Stitch cell directly onto row columns sequence
    row_parent.setChild(row_index, target_column, idn_item)

    # Force structural view lane expansion to show added items
    self.expandAll()

```

```

def save_to_db(self, db_path: str):
    """Passes model structure records and project configurations down to the persistence tier."""
    source_model = self._get_source_model()
    main_win = self.window()

    project_name = getattr(main_win, 'project_name', 'Untitled Project')
    # Extract live in-memory registry blocks from MainWindow context core
    project_settings = getattr(main_win, 'project_settings', {})

    IndexTreePersistence.save_to_db(source_model, db_path, project_name, project_settings)

def load_from_db(self, db_path: str):
    """Loads both tree structure and multi-column references while pausing UI sorting."""
    source_model = self._get_source_model()
    IndexTreePersistence.load_from_db(source_model, self, db_path)

def show_context_menu(self, position):
    index = self.indexAt(position)
    if not index.isValid():
        return

    menu = QMenu()
    remove_action = menu.addAction("Remove Entry")
    action = menu.exec(self.mapToGlobal(position))

    if action == remove_action:
        self._get_source_model().removeRow(index.row(), index.parent())

#--- Event overrides ---

def viewportEvent(self, event: QEvent) -> bool:
    """Intercepts viewport mouse hits, blocking native focus loops to eliminate jump bugs."""
    if event.type() == QEvent.Type.MouseButtonPress and event.button() == Qt.MouseButton.LeftButton:
        proxy_index = self.indexAt(event.pos())

        if proxy_index.isValid() and proxy_index.column() == 1:
            # Clear selections across columns
            self.selectionModel().clearSelection()

            # Calibrate click offsets metrics directly relative to column x box origin
            visual_rect = self.visualRect(proxy_index)
            click_x = event.pos().x() - visual_rect.x()

            # Clear tree focus explicitly right before firing navigation vectors
            self.clearFocus()

            self.referenceTokenClicked.emit(proxy_index, click_x)
            return True # Swallow layout clicks entirely!

        return super().viewportEvent(event)

def mousePressEvent(self, event):
    """Intercepts viewport clicks, blocking column 1 native events to stop jump bugs."""
    if event.button() == Qt.MouseButton.LeftButton:
        # Check which layout item index sits beneath the cursor coordinates
        index = self.indexAt(event.pos())
        if index.isValid() and index.column() == 1:
            # 1. Manually fire your custom selection model clearing block
            self.selectionModel().clearSelection()

            # 2. Safely forward the click event to your MainWindow connection slot
            self.clicked.emit(index)

            # 3. Accept the event and return early. This blocks native QTreeView row tracking loops!
            event.accept()
            return

    # Fall back to standard behaviors for column 0 right-clicks and navigation arrows
    super().mousePressEvent(event)

def leaveEvent(self, event):
    self.unsetCursor()
    super().leaveEvent(event)

```